

scripts/corpora

Summary of the code and the prompt styles in the horizon-free prompt corpora · Dr_Darveshi arc-coordinate pipeline

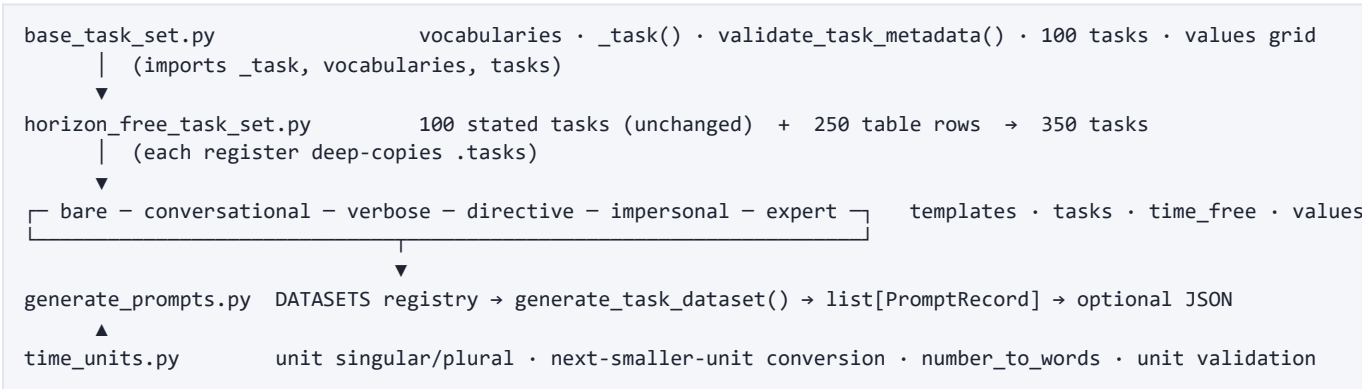
1. What this package is

`scripts/corpora` builds the **horizon-free prompt corpora**. These are planning prompts that name a task and **never state a time horizon** (no duration, deadline or amount of available time). They are the control condition: activations cached from them show what the model encodes about a planning task when no horizon is given. Researchers can then read a *task prior* (the horizon the model assumes on its own) and compare it with the horizons found in time-stated corpora.

The package has three layers:

Layer	Files	Role
Task metadata & task sets	<code>base_task_set.py</code> (2,340 lines) <code>horizon_free_task_set.py</code> (4,860 lines)	Define the controlled vocabularies, the <code>_task</code> constructor, 100 stated tasks, and the 350-task horizon-free set.
Prompt registers (6)	<code>bare_</code> , <code>conversational_</code> , <code>verbose_</code> , <code>directive_</code> , <code>impersonal_</code> , <code>expert_task_prompts.py</code>	Each file holds a list of templates and a copy of the task set, and sets <code>time_free = True</code> .
Generation engine	<code>generate_prompts.py</code> , <code>time_units.py</code> , <code>__init__.py</code>	Expands templates × tasks into <code>PromptRecords</code> , handles unit and number rendering, and provides the CLI.

2. Code architecture



2.1 base_task_set.py: the metadata schema

- **Controlled vocabularies**, stored as ordered tuples: `STAKES_LEVELS` (10 levels from *very_low* to *existential*), `TEMPORAL_HORIZONS` (8 bands from *seconds_minutes* to *generations*), `AGENCY_LEVELS` (10), `AFFECTED_SCOPES` (11), `PLANNING_TYPES`, `URGENCY`, `REVERSIBILITY`, `UNCERTAINTY`, `COORDINATION_LOADS`, `SCENARIO_TYPES` (17), `SCENARIO_REALISM`, and `PRIMARY_CONSEQUENCES` (14). Domains are open-ended and are collected from the task table.
- `_task(**kwargs)` is a keyword-only constructor that returns 19 labelled fields plus `units` (every supported unit). It also returns `stakes_score`, which is *derived* from `stakes` so the two can never disagree.
- `validate_task_metadata` runs at import time. It rejects any label outside its vocabulary and any empty `stakes_rationale`, so a typo cannot reach an analysis.
- **Three fields are kept apart on purpose.** `agency` is who has to act, `affected_scope` is who bears the consequence, and `temporal_horizon` is the *author's* hand-labelled expected duration. That label is never a model output and never filters the grid.

- There are 100 tasks: 50 `original` (wording frozen so cached activations stay valid) and 50 `continuum_expansion`, added to fill off-diagonal stakes × duration cells. The module also defines `values = [1, 2, 3, 5, 6, 10, 12, 15, 30]`, the grid used by the time-stated corpora.

2.2 horizon_free_task_set.py: the expanded control set

- **Why it is separate:** a stated corpus grows as tasks × 10 units × 9 values × templates, but a horizon-free corpus emits one prompt per (task, template). Tasks are cheap here, and the prior's sample size is the number of *tasks*.
- **Design:** it includes the 100 stated tasks unchanged (so priors stay comparable with earlier runs), then adds 250 `horizon_free_addition` tasks.
- **Style:** the new tasks are a positional tuple table (`_NEW_TASKS`) rather than keyword calls, so the stakes and horizon columns line up and gaps in the crossing are visible by eye. `_build_new_tasks()` zips each row with `_COLUMNS`, checks row length and duplicates, and then runs the same `_task` and `validate_task_metadata` functions. Collisions with the stated set raise at import time.
- It re-exports the vocabularies (for example `BANDS = TEMPORAL_HORIZONS`) along with `STATED_TASKS` and `DOMAINS` (60 domains).

2.3 generate_prompts.py: the generation engine

- **Types:** the `PromptRecord` TypedDict has the fields `text`, `template_id`, `template_metadata`, `task`, `task_metadata`, `base_value`, `base_unit`, `unit_variant`, `number_format`, `value`, `value_text` and `unit`. The horizon fields are `None` for every horizon-free record.
- **DATASETS registry:** maps each dataset name to its register module. Any module that exposes `templates`, `tasks` and `values` (and optionally `time_free` or `build_prompt_records`) can be registered.
- `generate_task_dataset()` has three paths:
 1. *Custom builder:* if the module defines `build_prompt_records`, the engine delegates to it. Template-level options then raise an error instead of being silently ignored.
 2. *Time-free* (all current registers): forces `remove_time_constraints` and emits one record per (template, task), or one random template per task when `randomize_template` is set.
 3. *Time-stated:* builds the full cartesian product of templates × tasks × units × values × {original, smaller-unit} × number formats.
- **Time-constraint stripping:** a template can supply an `unconstrained_template`. Otherwise, the `TIME_CONSTRAINT_LINE` regex removes lines labelled *Available time* / *Time budget* / *Deadline* / *Time constraint* / *Time window*.
- **Metadata passthrough:** every template key except `id`, `template` and `unconstrained_template` is copied into `template_metadata`, so register labels such as `perspective` and `request_style` reach the analysis automatically.
- **CLI:** `--dataset`, `--output-path`, `--randomize-template`, `--remove-time-constraints`.

2.4 time_units.py

This module supports the time-stated path. It supports 10 units (seconds to millennia) with singular forms, and a `NEXT_SMALLEST_UNIT` map (e.g. years→12 months, months→4 weeks, decades→10 years) used to create the "smaller" unit variant. It also provides `number_to_words` (0 to 999,999), `render_unit` (singular when the value is 1) and `validate_task_units`.

2.5 Recurring code conventions

- Relative imports fall back to absolute ones (`try: from . import ... except ImportError`), so a file works both as a package module and as a script.
- Each register copies its tasks with `_copy_task_config`, so the mutable `units` sets are never shared between modules.
- Already time-free templates set `unconstrained_template = template`, so time stripping can never alter them.
- `values = [1]` is kept only as a placeholder so every module has the same interface. Iterating a real grid would just produce identical duplicate prompts.
- Long docstrings explain the *experimental rationale* (which confound each choice controls for), not just the mechanics.

3. Prompt styles (registers)

All six registers share the same 350 tasks and differ only in wording, so an analysis can tell whether register (length, voice, framing or role) shifts the recovered horizon. `{task}` is always a lower-case verb phrase such as "abort a rocket launch after a sensor anomaly appears".

Dataset name	File	Register label	Templates	Records	Voice / distinguishing feature
conversational_no_time	bare_task_prompts	bare_task	3	1,050	Labelled field + fixed instruction
task_only	conversational_task_prompts	conversational	12	4,200	First-person, one-sentence questions
verbose_no_horizon	verbose_task_prompts	verbose	8	2,800	Long, discursive first-person requests
directive_no_horizon	directive_task_prompts	directive	6	2,100	Bare imperative addressed to the model
impersonal_no_horizon	impersonal_task_prompts	impersonal	6	2,100	Third person; no "I" and no "you"
expert_no_horizon	expert_task_prompts	expert	6	2,100	Generic professional role assigned first
Total			41	14,350	

Naming trap: the two original dataset names describe the *opposite* of their contents. `conversational_no_time` holds the **bare** prompts and `task_only` holds the **conversational** ones. `scripts.pipeline_config.REGISTERS` maps each dataset to its correct register label, so downstream code should always use that mapping, not the dataset name.

Bare task: conversational_no_time

Metadata: `prompt_framing` ∈ {task, goal, objective} · shortest, most schematic register

<code>task</code>	Task: {task}\n\nWrite a plan.
<code>goal</code>	Goal: {task}\n\nWrite a plan.
<code>objective</code>	Objective: {task}\n\nWrite a plan.

"Task: abort a rocket launch after a sensor anomaly appears 🚀🚀 Write a plan."

Conversational: task_only

Metadata: `perspective` (first_person; one second_person) · `request_style` (direct, walkthrough, hypothetical, reflective, optimization, prioritization, collaborative, advice, decomposition, conversational). These templates mirror the plain-English time-stated voices with the time clause removed.

<code>direct_help</code>	Could you help me plan how to {task}? Please keep the advice realistic.
<code>practical_walkthrough</code>	I need to {task}. Walk me through a practical way to pull it off.
<code>hypothetical_advice</code>	Imagine you had to {task}. How would you approach it?
<code>starting_now</code>	If I started now and wanted to {task}, what would a sensible plan look like?
<code>best_way</code>	What's the best way for me to {task}?
<code>what_to_focus_on</code>	I'd like to {task}. What should I focus on?
<code>think_it_through</code>	Help me think through how to {task}. Where should I begin and what should happen after that?
<code>external_request</code>	Someone has asked me to {task}. Can you suggest a realistic approach?
<code>break_it_down</code>	My goal is to {task}. How would you break that down?
<code>recommendation</code>	I'm trying to {task}. What would you recommend?

<code>how_to_go_about_it</code>	Say I wanted to {task}. How could I go about it?
<code>advice_request</code>	I could use some advice: I want to {task}. What's a sensible way forward?

Verbose: `verbose_no_horizon`

Horizon-free counterpart of the time-stated `plain_long` corpus (which had none): 8 voices, same IDs and metadata. It controls for **prompt length and elaboration**, since the other controls are all short. Each template adds 1–2 sentences asking for sequencing, checkpoints, priorities or pitfalls, without supplying a duration.

<code>direct_help</code>	Could you help me plan how to {task}? Please keep the advice realistic, practical, and grounded. I want a clear, actionable plan ... specific about what to do first, next, and last.
<code>practical_walkthrough</code>	I need to {task}. Walk me through a practical, step-by-step way ... preparation, key milestones, and potential pitfalls ... starting from scratch ...
<code>hypothetical_advice</code>	Imagine you had to {task}. How would you approach it? What order ... prioritize ... deliberately skip or postpone?
<code>starting_now</code>	If I started now and wanted to {task}, what would a sensible plan look like? ... clear checkpoints ... on track or falling behind ...
<code>use_the_time</code>	What's the best way for me to {task}? ... most efficient, effective use of my effort ... sequence tasks ...
<code>limited_availability</code>	I'd like to {task}. What should I focus on? ... highest-impact priorities ... left out or deferred.
<code>external_deadline</code>	Someone has asked me to {task}. Can you suggest a realistic approach, including how to pace myself ... avoid a frantic scramble at the end?
<code>break_it_down</code>	My goal is to {task}. How would you break that down into smaller, manageable steps or phases? ... how much each part should reasonably involve.

Directive: `directive_no_horizon`

`request_style` = imperative · `source_corpus` ∈ {named_horizon, directive}. The first four are the instruction half of the `named_horizon` prompts with the period sentence dropped; the last two are new.

<code>write_a_plan</code>	Write a plan to {task}.
<code>describe_how</code>	Describe how you would {task}.
<code>develop_a_plan</code>	Develop a plan to {task}.
<code>outline_realistic</code>	Outline a realistic plan to {task}.
<code>set_out_the_steps</code>	Set out the steps required to {task}.
<code>explain_how</code>	Explain how to {task}.

Impersonal: `impersonal_no_horizon`

`perspective` = third_person. This register removes the speaker entirely, which tests whether the readout tracks *who is asked* rather than *what is planned*.

<code>a_team_needs_to</code>	A team needs to {task}. What would a sensible plan look like?
<code>consider_the_problem</code>	Consider the problem of how to {task}. What does a workable plan involve?
<code>what_does_it_take</code>	What does it take to {task}?
<code>the_objective_is</code>	The objective is to {task}. Describe an approach that would achieve it.
<code>an_organisation_wants</code>	An organisation wants to {task}. How should the work be approached?
<code>suppose_someone_set_out</code>	Suppose someone set out to {task}. What would a reasonable plan cover?

Expert / role-assigned: `expert_no_horizon`

`assigned_role` ∈ {planner, project_manager, adviser, expert, operations_lead, reviewer} · `request_style` = role_assigned. The roles are **deliberately generic** and every task gets every role, because matching roles to tasks would reintroduce a task-dependent

confound.

experienced_planner	You are an experienced planner. A client wants to {task}. What plan do you propose?
project_manager	You are a project manager. Your team has been asked to {task}. How do you plan the work?
trusted_adviser	You are a trusted adviser. Someone has come to you wanting to {task}. What do you advise?
domain_expert	You are an expert in this area. Explain how to {task}.
operations_lead	You lead operations for an organisation that must {task}. Set out your plan.
independent_reviewer	You have been asked to review a proposal to {task}. What should the plan contain?

How the registers vary

Axis	Bare	Conversational	Verbose	Directive	Impersonal	Expert
Length	very short	short	long	very short	short	short-medium
Speaker	none	"I"	"I"	none	none	none
Addressee	implicit "you"	"you"	"you"	"you" (imperative)	none	"You are a ..."
Form	labelled field	question	question + elaboration	command	question / description	role + scenario + ask
Actor in scenario	—	self	self	model	team / org / someone	client / team / org

4. The task set at a glance

source_set	Tasks	Purpose	Distribution	Counts (of 350)
original	50	Wording frozen; used by stated corpora	planning_type	systems 109 · procedural 58 · crisis_response 50 · project 40 · strategic 35 · tactical 33 · creative 13 · none 12
continuum_expansion	50	Off-diagonal stakes × duration cells	agency	individual 111 · organization 72 · multi_org 38 · global_coord 30 · intl_coalition 26 · national_gov 25 · local_gov 14 · community 13 · household 12 · small_team 9
horizon_free_addition	250	Larger prior sample; sweeps stakes within each band	scenario_realism	realistic 235 · plausible_hypothetical 67 · speculative 43 · fictional 5
Total	350	60 domains		

Example tasks by stakes and horizon. *answer a yes-or-no question (very_low, seconds_minutes) · prevent further escalation after limited nuclear weapon use (near_existential, seconds_minutes) · write an encyclopaedia of an invented world (low, years_decades) · build a planetary defence capability against asteroid impact (near_existential, years_decades).*

Stakes × temporal_horizon crossing (all 350 tasks)

stakes \ horizon	sec-min	min-hr	hr-day	day-wk	wk-mo	mo-yr	yr-dec	gen.	Σ
very_low	24	0	1	0	0	0	0	0	25
low	7	0	8	10	4	6	4	1	40
medium_low	0	0	5	6	4	3	4	1	23
medium	0	0	6	4	4	10	5	5	34
medium_high	1	2	4	4	5	10	6	6	38
high	14	2	5	5	1	8	10	11	56
very_high	7	3	6	12	1	7	14	9	59
catastrophic	6	0	5	14	2	2	12	1	42
near_existential	4	0	0	2	1	5	6	9	27
existential	1	0	1	0	0	1	0	3	6
Σ	64	7	41	57	22	52	61	46	350

The off-diagonal cells (for example high stakes resolved in seconds, or low stakes pursued for generations) are filled, which is what separates a stakes effect from a duration effect. Two gaps remain: the **minutes-hours** band has only 7 tasks and **weeks-months** has 22, and **very_low** stakes appear almost only in the seconds-minutes band (24 of 25). `pipeline_config.DEFAULT_STAKES_MERGES` merges *near_existential*→*existential* and *medium_low*→*medium* for analysis.

5. Observations & loose ends

- **Stale docstring references.** Several docstrings cite modules from an earlier package layout that no longer exist here (`nu_d.corpora.structured_time_prompts`, `conversational_time_prompts`, `verbose_time_prompts`, `named_period_prompts`). The time-stated corpora they describe are not in `scripts/corpora`.
- **Broken fallback import.** `base_task_set.py` falls back to `from utils import SUPPORTED_UNITS`, but that module does not exist. The code only works through the package-relative import from `time_units`.
- **Leftover template IDs in the verbose register.** `use_the_time`, `limited_availability` and `external_deadline` keep their time-stated IDs, which were kept so records match across corpora. Their text is time-free, but phrases like "*pace myself*" and "*frantic scramble at the end*" still hint at time pressure.
- **Duplicated boilerplate.** Every register module repeats `_copy_task_config`, the `unconstrained_template` loop, `time_free = True` and `values = [1]`. `conversational_task_prompts.py` is also missing `__all__`, which the other five define.

- **Unused paths in the engine.** The time-stated path, the `build_prompt_records` hook and `time_units.py` are fully implemented, but no module registered in `DATASETS` uses them today.